

set_os

REFERENCE

API consumer

initiate an over-the-air firmware update on the reader

Overview

`set_os` initiates an over-the-air (OTA) firmware update — it points the reader at a firmware `.DAT` image at a URL and triggers the download-and-flash cycle. It is a state-changing command on the device-management ([MGMT](#)) endpoint, and it is both **asynchronous** and **session-severing**: the device accepts the payload, downloads and flashes the firmware, and **reboots** — dropping off the network for the duration. The authoritative confirmation of the outcome is an out-of-band `get_version` read-back.

`set_os` **reflashes and reboots the reader**. The reply `code 1` is an accept (returned for success and failure alike), and the reader rejects every other command with `code 4` while the update runs. See the danger callout in [Notes and Caveats](#); confirm the result with `get_version`.

Details

Property	Value
Interaction Pattern	Asynchronous Request → Accept → Reboot → Verify (completion confirmed out-of-band)
State Change	STATE-CHANGING (downloads and flashes firmware; reboots the reader)
Reversibility	IRREVERSIBLE (no single-step undo; re-flash a prior build with another <code>set_os</code> to roll back)
Required Request Fields	<code>command</code> , <code>requestId</code> , <code>OSUpdateDetails</code> (with <code>url</code> and <code>authenticationType</code>)
Applies To	Endpoint: Management (MGMT). Reader: RFD40 series, RFD90 series
API Version	<code>v1.1</code> (API Version Reporting)
Related	<code>get_version</code> , <code>install_certificate</code> , <code>reboot</code> , <code>get_status</code>

When to Use

Use `set_os` to **update the reader's firmware** — during commissioning, a fleet migration, or a security/feature rollout.

- **Host the image** — place the firmware `.DAT` at a URL the reader can reach.
- **Choose authentication** — `NONE` for an open/internal server, or `CERTIFICATE` (with a CA reference or inline PEM) plus a `verificationType` for a secured HTTPS download.
- **Check preconditions** — ensure adequate battery and flash first (low battery → [code 14](#), insufficient flash → [code 8](#)).
- **Verify afterward** — always confirm the new build with `get version`.

Topic Mapping

Message	Direction	Topic	Note
Request	App → Reader	<code><tenantId>/MGMT/clients/cmnd/<serial></code>	On the management endpoint.
Response	Reader → App	<code><tenantId>/MGMT/clients/resp/<serial></code>	The code 1 accept, and the code 4 lock returned to <i>other</i> commands during the install.
Event	Reader → App	<code><tenantId>/MGMT/clients/event/<serial></code>	The <code>FIRMWARE_UPDATE</code> notifications (start, progress, outcome). See alerts .

Topics follow the pattern `<tenantId>/<topic>/<serial>` — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the reader's serial number, which the reader appends itself in its model-prefixed form (for example `RFD40-212735201D0053`). `App → Reader` means your app publishes and the reader subscribes; `Reader → App` means the reader publishes and your app subscribes.

See [Topics and Channels](#).

Key Fields

- `OSUpdateDetails.url` (request) — the firmware `.DAT` download URL (a bad/unreachable URL → [code 9](#)).
- `OSUpdateDetails.authenticationType` (request) — `NONE` or `CERTIFICATE`; for `CERTIFICATE`, supply `caCertificateFile` (installed reference) or `caCertificateFileContent` (inline PEM).

- `OSUpdateDetails.verificationType` (request) — TLS verification of the download server; we recommend setting it for HTTPS downloads (see [Request Semantics](#)).
- `response.code` (response) — 1 means *accepted*, not *updated*; confirm with `get_version`. See [Response Codes](#).

Request Payload

The envelope is `command`, `requestId`, and the `OSUpdateDetails` payload (`url` + `authenticationType`). Pick an **update source** below; the Example and Schema update together. Use the **Schema** tab for the full field reference and the **Example** tab for the concrete payload.

Update source

No authentication HTTP

Request payload

Example: authenticationType NONE · HTTP

```
1 {
2   "command" : "set_os" ,
3   "requestId" : "abc123" ,
4   "OSUpdateDetails" : {
5     "url" : "http://192.168.29.39:8000/Build-3.10.27/Firmware/PAAFKS00-012-R
6     "authenticationType" : "NONE" ,
7     "verificationType" : "VERIFY_NONE"
8   }
9 }
```

Request payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
1	command *	string	—	set_os
2	requestId *	string	—	abc123
3	OSUpdateDetails *	object	—	—

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
4	url*	string	uri	https://192.168.29.39:8000/Build-3.10.27/Firmware
5	authentication*	enum	NONE CERTIFICATE	CERTIFICATE
6	verificationType	enum	VERIFY_NONE VERIFY_HOST_AND_PEER	VERIFY_NONE

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
7	certificate	string	—	<pre> -----BEGIN CERTIFICATE----- \nMIIDPTCCAiWgAwIBAgIUHtj1WDceNR9 1Ne7sFmHkwDQYJKoZIhvcNAQEL\nBQAwI A1UEAwwGVGVzdENBMQ4wDAYDVQQKDAVaZ MAkGA1UEBhMC\nVVMwHhcNMjYwNTExMTM cNMzYwNTA4MTMxNDQ3WjAuMQ8wDQYDVQQ ZXN0Q0ExDjAMBGNVBAoMBVplYnJhMQswC EwJVUzCCASIwDQYJKoZIhvcN\nAQEBBQA CCAQoCggEBANR8E/mxoQ5uTPW/muWnGoW lIiJiCM/n92\ndMLIQ+TVgb9mcOz80Vnpg ZaIG4CtnBufyIJvNKid+mgrrrFh18HtMnE oPC4M469Rm+0oEN9GkUHgtgF2N4Qt9bYo yPlFuUOC+uaYRCCn3Hm8ko\nXE/a7XWdX Mzzf7MoNq14AfX431RKgvpMgAqZTnDWtm q0t64tP7\naKIAJV2avJd87dF8x+K80kL uNFpXN7KoC7BBwdTzbG8VWYNUZeG8eCqB Y5Ik7POMR0AM/TxRBLAXyoZHSPdEpEoaQ Hv0CAwEAAaNTMFEwHQYD\nVR0OBBYEFJH FYnVcf2++X9N8UlkahMB8GA1UdIwQYMBa XPeyFY\nnnVcf2++X9N8UlkahMA8GA1UdE MAMBAf8wDQYJKoZIhvcNAQELBQADggEB\ MhuQ9sQLFE9wiGw4BIILxnX30FbphWM8/ sH/NOWpjuLG+ZAxe8n\nn+QUhaU9hruBEa vZ9NtUumcAmAh3s29KD4NpjwglqUYfKQp MXuW\nnzCmqlAqGVzK3ve/Fuurgdbk3kAp w629dmmRhoP+2sdCYMAuky348/4G1\nnY ygC4/WCfWg/7XaeQ+6kqKtY6ijSJFFCNw 3XAqP+kZ0e2taL5a\nnrv5UUPRMGCs20J 8uPOICNWUER3DbqAJ4jXKhSHdsG6fNDyJ 0j\nVqh5mX6k2PZe0eylcukyu/s=\n--- CERTIFICATE----- </pre>
8	certificate	string	—	/apps/ca_cert.pem

Request Semantics

Modes. `authenticationType` selects how the reader authenticates to the firmware source: `NONE` for an unauthenticated download (an open or internal server), or `CERTIFICATE` to authenticate the HTTPS download against a CA — supplied either as a **pre-installed** reference (`caCertificateFile` , installed via `install_certificate`) or **inline** (`caCertificateFileContent`), and paired with a `verificationType` for TLS server verification. Both `CERTIFICATE` forms are in the [Request Payload](#) selector: **Pre-installed CA certificate HTTPS** and **Inline CA certificate HTTPS**.

- **Authentication & transport security.** For `authenticationType: CERTIFICATE` , supply the CA either as an installed reference (`caCertificateFile` , pre-installed via `install_certificate`) or inline (`caCertificateFileContent`); if `caCertificateFileContent` is empty, the reader uses an already installed filestore certificate. `authenticationType: NONE` means no authentication is needed to access the URL. `verificationType` sets how far the reader validates the download server's TLS certificate: `VERIFY_HOST_AND_PEER` checks the certificate chain and that the certificate matches the host; `VERIFY_NONE` does no verification and must not be used in production.
- `OSUpdateDetails` **is required**. The contract requires it, so a request without it is invalid rather than merely unable to update. A bad or unreachable `url` returns [code 9](#) .

Response Payload

The immediate reply is an **accept** — `command` , `requestId` , `apiVersion` , and `response` with [code 1](#) . It confirms receipt, not a completed update.

Response payload

Example: *accept*

```
1  {
2      "command" : "set_os" ,
3      "requestId" : "abc1234" ,
4      "apiVersion" : "V1.1" ,
5      "response" : {
6          "code" : 1 ,
7          "description" : "Command payload is accepted"
8      }
9  }
```

Response payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	string	—	set_os	Echoes the executed command.
2	requestId *	string	—	abc123	Echoes the request's requestId — pairs reply to request.
3	apiVersion *	enum	V1.0 V1.1 V1.2 V1.21 V1.22	v1.1	API version reported by this command. Emitted per-handler, not device-wide.
4	response	object	—	—	Accept/status envelope. Not in the contract's required set for this reply, although every published example carries it. This is an accept, not a terminal result (see the lifecycle below).
5	code *	integer	0–101	0	1 = the payload was accepted (the accept code for this command — not 0). See Response Codes below.
6	description *	string	—	Success	Human-readable status.

Lifecycle

`set_os` is an accept-then-verify cycle, not a single request/response.

Acknowledgment

The device replies `response.code 1` ("Command payload is accepted") on the response topic. This confirms receipt only, and is **identical for success and failure**.

Progress

A `FIRMWARE_UPDATE` notification appears on the event topic as the download/flash begins. The block below is the example the schema declares for it, not an observed one:

Event — FIRMWARE_UPDATE start (schema example)

```
1  {
2    "type": "NOTIFICATION",
3    "timestamp": "2026-06-18T11:30:50.387Z",
4    "state": "ONESHOT",
5    "id": "FIRMWARE_UPDATE",
6    "priority": "HIGH",
7    "alertDetails": {
8      "fwUpdateStatus": {
9        "updateStatus": "Starting FW update"
10     }
11  }
12 }
```

Notifications continue on the same topic while the install runs. Each of the five recorded during one update observed on an RFD40 carried `alertDetails.fwUpdateStatus.overallProgress` — percent of the update completed — beside a human-readable `updateStatus`. `overallProgress` is optional and the start example above carries none, so treat its absence as unknown rather than as zero. Three of those five notifications are shown here and under [Confirmation](#) below, across the 15.265 s in which `overallProgress` ran from 87% to 100%; the earlier part of that update was not recorded.

Event — FIRMWARE_UPDATE in progress, 87% (live)

```
1  {
2    "type": "NOTIFICATION",
3    "timestamp": "2026-07-07T11:49:50.628Z",
4    "state": "ONESHOT",
5    "id": "FIRMWARE_UPDATE",
6    "priority": "HIGH",
7    "alertDetails": {
8      "fwUpdateStatus": {
9        "updateStatus": "In progress",
10       "overallProgress": 87
11     }
12  }
13 }
```

The identical `In progress` value arrived twice — at 87%, then at 95% 10.014 s later — so only the first is shown. The notification after it carried a status string naming a component file inside the firmware package:

Event — FIRMWARE_UPDATE in progress, 98% (live)

```
1  {
2    "type" : "NOTIFICATION" ,
3    "timestamp" : "2026-07-07T11:50:04.424Z" ,
4    "state" : "ONESHOT" ,
5    "id" : "FIRMWARE_UPDATE" ,
6    "priority" : "HIGH" ,
7    "alertDetails" : {
8      "fwUpdateStatus" : {
9        "updateStatus" : "Updating file PAAFKS00-013-R04L0.DAL" ,
10       "overallProgress" : 98
11     }
12   }
13 }
```

- `updateStatus` is display text, not a machine token. It carries whatever the firmware shows an operator, file names included, and four of the five recorded notifications carried a value that is not one of the five `fwUpdateStatus.updateStatus` declares — only the terminal `FW update success` matched. The declared lowercase values `updating`, `skipping` and `failed` appeared in none of them: the reader sent sentences such as `Updating file ...` and `Skipping file ...` instead, so the obvious test `updateStatus === 'updating'` matches nothing this reader sends. Render the string as opaque text and drive logic from `overallProgress` and from the terminal literal under Confirmation below. The schema remains normative; this is an observed device deviation, cataloged in [Known device deviations from the schema](#).
- **A status string can arrive incomplete.** The notification at 99% reported `Skipping file ...`, naming a path inside the firmware package, and the value ended mid-name, its final character recorded as an encoding-error artifact rather than as text. Whether the reader truncated the string or the recording chain corrupted it cannot be told from a single sample, and no maximum length can be inferred from one either: do not assume a length limit, and do not substring-match. `overallProgress` in that same notification was a well-formed `99`, which is the practical case for branching on the number rather than on the text.
- **The stream's timing is not a contract.** Across the recorded window the gap between notifications ran from 10.014 s down to 0.284 s while `overallProgress` advanced by 8, then 3, then 1, then 1 percentage points — neither a fixed interval nor a fixed increment, so do not derive a silence timeout from it. Emission is gated by `eventConfiguration.firmwareUpdate` in [config_events](#) and can be switched off altogether. The stream therefore gives you no deadline, so bound the wait on your own clock: set a wall-clock budget for the whole update from your own fleet's measurements, and on expiry poll [get_version](#) until the reader answers. Treat silence as unknown, never as failure.

Session Impact

While the update runs, the reader rejects **every other command** on the response topic with `code 4` ("Firmware update in [progress](#)"), echoing the blocked command's name:

Response — another command blocked during the update (code 4)

```
1  {
2    "command" : "get_endpoint_config" ,
3    "requestId" : "abc123" ,
4    "apiVersion" : "V1.1" ,
5    "response" : {
6      "code" : 4 ,
7      "description" : "Firmware update in progress"
8    }
9  }
```

The reader reflashes and reboots, and is briefly off the network.

Confirmation

The reader also reports the outcome on the event topic. The terminal notification carries

`updateStatus: FW update success` with `overallProgress: 100`:

Event — FIRMWARE_UPDATE success, 100% (live)

```
1  {
2    "type" : "NOTIFICATION" ,
3    "timestamp" : "2026-07-07T11:50:05.893Z" ,
4    "state" : "ONESHOT" ,
5    "id" : "FIRMWARE_UPDATE" ,
6    "priority" : "HIGH" ,
7    "alertDetails" : {
8      "fwUpdateStatus" : {
9        "updateStatus" : "FW update success" ,
10       "overallProgress" : 100
11      }
12    }
13  }
```

That literal is the one `updateStatus` value worth an equality test — it is a declared enum member and the observed notification matched it exactly — with `overallProgress: 100` as corroboration rather than as the test, because a single observation cannot establish that 100 always means success. Every notification recorded, the terminal one included, carried `state: ONESHOT`, so the terminal signal sits in `fwUpdateStatus` and not in the envelope: **do not wait for a** `CLEAR`.

Gate on the read-back, not on the notification. Run `get_version` and compare `readerVersion.firmwareVersion` against the target build (see the version note below). The notification tells you when to stop waiting; the read-back tells you whether the firmware changed. A failed update leaves the prior version in place; terminal failure is reported as `fwUpdateStatus.updateStatus: failed` in the `FIRMWARE_UPDATE` event (which carries no response code), not on the `set_os` accept. `failed` is the value the schema declares for that case; no failure arose in the traffic behind the [examples](#) above, and because two of the three bare-stem values the schema declares arrive as sentences on this firmware, a real failure may well arrive as a sentence too — so detect failure by the read-back, not by matching `failed`.

Response Codes

`set_os` returns `response.code`. The codes applicable to this command are below; for the full table see the [Response Codes](#) and [Correlation and Request/Response Pairing](#).

Code	Description
1	Command payload is accepted (<code>IOT_STATUS_CMD_PAYLOAD_ACCEPTED</code>) — the accept code for this command . The update proceeds asynchronously; verify with <code>get_version</code> .
0	Success — terminal success; in practice <code>set_os</code> acks with 1, and completion is confirmed via the <code>get_version</code> read-back.
4	Firmware update in progress (<code>IOT_STATUS_FW_UPDATE_IN_PROGRESS</code>) — returned to other commands while an update is running (the device-wide lock). Pause orchestration until the device returns.
8	Insufficient flash size (<code>IOT_ERROR_LOW_FLASH_SIZE</code>) — not enough free flash to download the firmware. Free space or use a smaller package.
9	File not found (<code>IOT_STATUS_FILE_NOT_FOUND</code>) — the firmware <code>url</code> is wrong or unreachable.
13	<code>Firmware update Failed</code> (<code>IOT_ERROR_FW_UPDATE_FAIL</code>) — the defined failure code. In practice <code>set_os</code> acks only with 1; a failed update is reported as <code>fwUpdateStatus.updateStatus: failed</code> in the <code>FIRMWARE_UPDATE</code> event (which carries no response code), not on the <code>set_os</code> response — but do not equality-test that literal; see Confirmation .
14	Battery is low, Cannot update firmware (<code>IOT_ERROR_FW_UPDATE_FAIL_LOW_BATTERY</code>) — charge or connect external power, then retry.

Code	Description
101	Default error — generic catch-all.

Gate success strictly on the `get_version` read-back, not on the `set_os` response code — the reply `code 1` acknowledges only that the payload was accepted, and is returned for both success and failure.

Examples

Publish `set_os`; the device acks with the `code 1` envelope under [Response Payload](#), then runs the [Lifecycle](#). The `CERTIFICATE` variants are in the [Request Payload](#) selector; the `FIRMWARE_UPDATE` notifications — start, in-flight progress, and terminal success — and the `code 4` lock are shown under the [Lifecycle](#) above.

Notes and Caveats

⚠ REFLASHES AND REBOOTS THE READER — PLAN FOR UNREACHABILITY

The device goes dark and locks out other commands. While the update runs, the reader rejects **every** other command with `code 4` ("Firmware update in **progress**") and is then unreachable across the reboot. Do not retry-storm — pause orchestration until the device re-attaches. The reply `code 1` is returned for **both** success and failure, and there is no terminal result on the response topic: confirm the outcome with `get_version` and compare the firmware version.

- **Preconditions.** Ensure the device is ready before issuing: low battery → `code 14`; insufficient flash → `code 8`.
- **Confirm by full version string.** `readerVersion.firmwareVersion` is a free-form string. Compare the **whole** value against the target build — the build-variant prefix can change (e.g. `PAAFKS00-013-R02` → `SAAFKS00-013-R02`), so match the full string rather than a substring. See `get_version`.

Sequence a firmware rollout as: pre-check with `get_status` (power/battery) → `set_os` → wait out the reboot (expect `code 4` on other commands) → confirm with `get_version`. For the concept-level view, see [Device Lifecycle](#).

Related:  [Device Lifecycle](#) ·  [get_version](#) ·  [install_certificate](#) ·  [reboot](#) ·  [Response Codes](#)